

Runcell: A Gamified and Privacy Preserving Learning Layer Inside JupyterLab

Turning a course's own notebooks into verified exercises, and class telemetry into anonymous teaching signal

YUNES MAHAN, Ludwig-Maximilians-Universität München, Germany

Computational notebooks are the default medium for teaching data science, yet they are pedagogically passive: a student can scroll a finished notebook, run every cell, watch correct output appear, and finish having practised nothing. Notebook tooling for education is dominated by assessment infrastructure such as nbgrader, which marks submissions well but does not change what happens while the student reads. We present *Runcell*, a JupyterLab 4 extension that turns a course's existing notebooks into an active learning experience without asking teachers to rewrite their material. Runcell derives three kinds of exercise from each code cell using a large language model, verifies answers by executing them on the live Jupyter kernel and comparing runtime output rather than source text, and layers experience points, an opt in leaderboard and consent based peer comparison on top. Its instructor dashboard is built on *anonymous* per cell attempt records, so teachers see which cells the class fails without being able to attribute a failure to a named individual. Our contributions are an output equivalence verification loop that admits any working solution instead of one canonical answer, a privacy asymmetry in which motivational features are identified and diagnostic features are anonymous by construction, and a degradation design in which the system stays usable with no AI key and no backend.

CCS Concepts: • **Applied computing** → **Interactive learning environments**; *Computer-assisted instruction*; • **Human-centered computing** → *Human computer interaction (HCI)*.

Additional Key Words and Phrases: computational notebooks, JupyterLab, gamification, programming education, large language models, learning analytics, privacy by design

Project report for Practical Intelligent Interactive Systems, Sommersemester 2026.

1 INTRODUCTION

Jupyter notebooks are the standard delivery format for teaching data analysis, scientific computing and applied machine learning [9]. A typical week ships slides and notebooks in which the instructor has already written the code, already executed it and already saved the output. The intent is that the student reads the narrative, runs the cells and internalises the reasoning. The observed behaviour is different: because the notebook arrives complete, the cheapest path through it is to press Shift and Enter repeatedly, watch plots appear and reach the last cell in minutes. The student has produced a perfect artifact and has practised nothing.

This is not hypothetical. Roughly one in four notebooks on GitHub contains no prose at all, and among academic notebooks most describe what was done without discussing why or what the result means [13]; a large fraction do not even execute reproducibly in order [11]. The medium optimises for exploration and for presenting a finished result, neither of which is optimising for learning.

The learning sciences explain what is missing. Retrieval practice, producing an answer rather than rereading it, yields substantially better long term retention than restudying [12]; explaining a step to oneself improves comprehension and transfer [4]; and feedback works best when it is immediate, task specific and tells the learner how to close the gap [8]. A pre executed notebook offers recognition instead of recall, presentation instead of explanation, and no feedback loop at all, because there is nothing the student can get wrong.

Existing notebook tooling addresses a different problem. nbgrader, the reference solution, is an assignment and grading pipeline [1]: author a master notebook with hidden tests, release a stripped student version, collect and auto-grade. It solves assessment well, but it does not change the experience of reading course material, and every exercise must still be written by hand with its own tests.

Problem statement. *How can a course's existing, unmodified notebooks be turned into an active, feedback rich learning experience, without requiring the instructor to author exercises by hand, and without turning the resulting telemetry into a surveillance instrument aimed at individual students?*

Runcell answers that as a JupyterLab 4 extension with two modes, *Learn* and *Explain*, described in Section 5. Behind both sits a course backend that tracks progress, ranks students on an opt in leaderboard and feeds an instructor dashboard built exclusively on anonymous aggregates.

2 BACKGROUND AND RELATED WORK

Assessment tooling. nbgrader [1] defines the state of practice, and variants such as Otter Grader follow the same batch shape. Two properties matter for our positioning: feedback is delayed until grading, the opposite of what the feedback literature recommends [8], and correctness is defined by instructor written unit tests, so authoring effort scales with the amount of material. Runcell does not compete on grading; it targets the reading phase, and derives exercises rather than requiring them to be written.

Gamification in programming education. Sailer and Homner [15] report small but real positive effects of gamification on cognitive ($g = 0.49$), motivational ($g = 0.36$) and behavioural ($g = 0.25$) outcomes. Crucially the effect depends on which elements are used: reward and status mechanics bolted onto unchanged content perform noticeably worse than designs adding genuine challenge. Self determination theory explains why [14]: points and rankings are controlling extrinsic regulators that can crowd out intrinsic motivation when they are the only change. We therefore did not gamify reading. We changed the activity from reading to solving and only then attached points, and we made the leaderboard opt in, because a ranking a student cannot decline is exactly the mechanic the literature warns about.

The closest prior systems come from the Learning Technologies group at RWTH Aachen, who integrate game elements into Jupyter driven by learning analytics [2, 3], logging JupyterLab interaction events into a Learning Record Store via xAPI and using that stream as the substrate for game elements. The difference is where the game lives. In their design the notebook is unchanged and the gamification observes interaction with it, so a student who runs every cell without thinking still generates activity; in Runcell the notebook itself is transformed, so points cannot be earned without producing a correct artifact. We also add a course model, instructor authoring and an analytics surface.

Exercise types and authoring cost. Parsons problems, in which a student reassembles scrambled code fragments, are the best studied alternative to free code writing. Reviews [6, 7] report that they reduce working memory load, in line with cognitive load theory [18], and reach outcomes comparable to equivalent code writing exercises in less student time; the caveat is that authoring good problems with plausible distractors is expensive. Runcell borrows the underlying idea, that a spectrum of scaffolding between reading code and writing it from nothing is valuable, and derives each of its three exercise types from the cell the instructor already wrote.

Large language models in computing education. Sarsa et al. [16] showed that an LLM can generate programming exercises with sample solutions and test cases at a quality that is usable but not uniformly reliable, so generated artifacts need verification before students see them. CodeHelp [10] deployed an LLM assistant with guardrails preventing it from emitting full solutions, and found it complemented rather than replaced human help. Denny et al. [5] set out the broader framing: because models solve introductory assignments outright, assessment must move towards process, comprehension and explanation. Runcell therefore treats the AI as a content generator and tutor rather than a gatekeeper, and never trusts its answer key.

Learning analytics and its ethics. Instructor dashboards showing per student progress are standard, and they are also the part of learning analytics that draws the most serious objections. Slade and Prinsloo [17] catalogue the dilemmas: data ownership, what informed consent means when participation is compulsory, and the tension between care and surveillance. An early iteration of our system had a per student drill down showing an individual's attempts cell by cell; it was removed after a formative review whose strongest message was that instructors should see *where the class fails* without seeing *who failed*. To our knowledge the resulting asymmetry, in which the motivational subsystem is identified and consensual while the diagnostic subsystem is structurally anonymous at the storage layer, is absent from the systems surveyed above.

3 DESIGN GOALS

The analysis above, plus a formative review of an early prototype, produced six goals we treat as requirements and justify later decisions against.

- G1. Zero authoring cost.** A teacher publishes an ordinary notebook and exercises exist for it immediately, while keeping the ability to override any generated exercise by hand.
- G2. Correctness by execution.** Any code producing the required result must be accepted. There is more than one correct way to write almost every cell, and rejecting a working solution for not matching a reference string is both wrong and demoralising.
- G3. Immediate, task level feedback.** Following [8], the student learns within seconds whether they are right, and when wrong receives a graduated nudge rather than the answer.
- G4. Motivation without coercion.** Points are on by default because they attach to genuine challenge; social comparison is opt in, because a mandatory ranking is the mechanic that [14, 15] call counterproductive.
- G5. Diagnostic value without surveillance.** The instructor gets class level failure signal, and the storage layer must make individual attribution impossible rather than merely hidden.
- G6. Graceful degradation.** The system stays useful with no AI key, no backend and no course at all.

4 SYSTEM ARCHITECTURE

Runcell is a JupyterLab 4 extension in three layers (Figure 1). Building an extension rather than a separate web application follows from G2: verification needs a real Python kernel with the course libraries installed, and inside JupyterLab that kernel already exists. A standalone application would have had to provision and sandbox its own execution environment.

The frontend is about 16,000 lines of TypeScript across 51 modules, registering one JupyterLab plugin that boots a login overlay, fetches configuration from the server extension and mounts a full screen widget into the shell. Screens are plain DOM modules with a shared design system rather than a component framework, since adding React to an

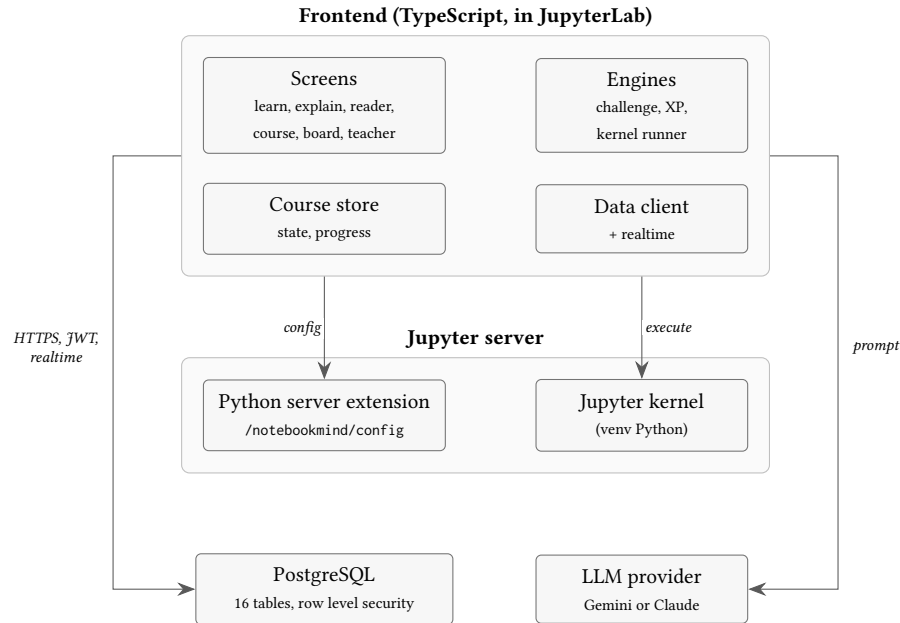


Fig. 1. Three layer architecture. The frontend talks to three independent services: the kernel for verification, the LLM for generation, PostgreSQL for state. Any can be absent without taking the others down.

extension bundle materially increases build complexity and size for a project whose interactivity is mostly local to individual cards. Code editing uses CodeMirror 6 with the Python mode, the editor JupyterLab itself uses.

The Python side is thin: a server extension exposing one endpoint that reads a local `.env` and returns the LLM key and the database credentials. It exists because a browser extension has no filesystem access, so without it every user would have to paste API keys into the interface.

State lives in a PostgreSQL database on Supabase, accessed directly from the browser over HTTPS with a JSON Web Token issued at sign in. There is no bespoke application server: a REST API in Python would have meant reimplementing authentication, session refresh, authorization and websocket fan out. Instead authorization is expressed once as row level security policies in the database, enforced for every query regardless of client, and live updates arrive over Supabase Realtime, which streams the logical replication feed to subscribers while applying the same policies. The security perimeter is the database, not the client. The schema is sixteen tables and eight stored procedures over twenty one ordered migrations.

5 THE LEARNING EXPERIENCE

5.1 Learn mode: three exercise types

In Learn mode the notebook is not shown. The student is walked through it one code cell at a time, and each cell is presented as one of three exercise types (Table 1).

The three types form a scaffolding ladder in the sense of the Parsons literature [7]. Comprehension imposes the least generative load, since code and output are both visible and the task is to reason about effect. Fix the bug imposes more: the student must hold a correct mental model of the cell and locate where the code diverges from it. Write the

Table 1. Exercise types and how answers are verified. Free text prediction was implemented and then disabled, because grading it requires trusting the model.

Type	Student produces	Verified by
Fix the bug	A corrected version of code containing exactly one planted defect	Kernel execution, output compared to the original cell
Write the cell	The whole cell from scratch, given a written brief	Kernel execution, output compared to the original cell
Comprehension	One of four options describing what the already executed cell does	Match against the generated answer key

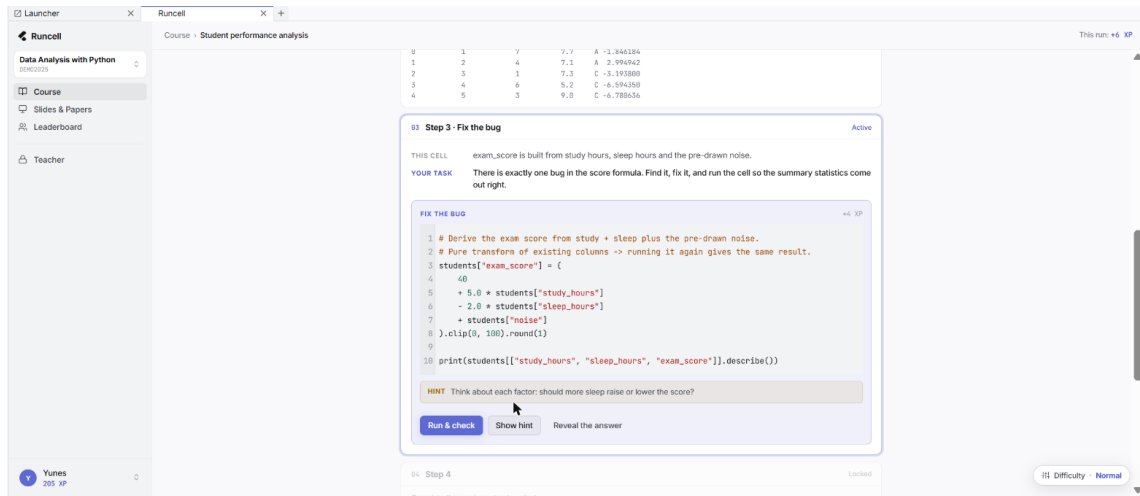


Fig. 2. Learn mode. The brief names the actual variables the cell touches, and Run and check executes the answer on the live kernel.

cell imposes the most, since the editor starts empty and the student reconstructs the computation from a brief. That is retrieval practice in the strict sense [12], and it is the fallback type.

Each exercise carries a difficulty label from easy to impossible, which sets the reward at 2, 4, 6 or 10 experience points; finishing a notebook is worth 30. The values are not empirically tuned. They exist to make harder work visibly more valuable, the property that matters for the challenge oriented gamification [15] finds effective.

5.2 Generation and verification

Figure 3 shows the loop. Before the session starts, Runcell executes the original notebook cell by cell on a fresh kernel and records the normalised output of every cell. That is the reference. Generation then sends the cell source to the LLM with a schema constrained prompt naming the three permitted types, forbidding free text answers, and demanding that the brief reference the real objects in the cell rather than generic filler.

The reply is never used as returned. It passes through a coercion function that repairs or rejects it: an unknown type is replaced, a fix the bug exercise that arrived without buggy code is rejected, a multiple choice exercise with fewer than two options is rejected, and a write the cell exercise that arrived pre populated is emptied. Any rejection falls through to a deterministic exercise built from the source alone. This is our operational answer to the reliability caveat

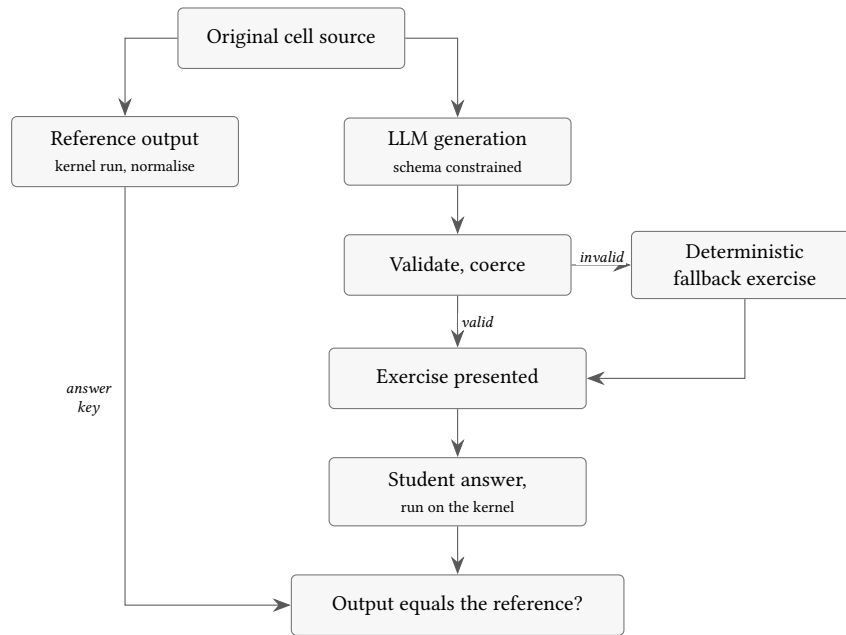


Fig. 3. Generation and verification. The model only proposes the task; the answer key is the output the instructor’s own cell produced, so any working solution is accepted and a badly generated exercise degrades to a safe fallback. A failed comparison returns the student to the exercise with the next hint unlocked.

in [16]: we constrain the output shape, validate it mechanically, and make the failure mode a safe exercise rather than a broken one.

Verification is the part we consider strongest. When the student presses Run and check, their code executes on the same kernel that produced the reference, and the run is accepted if and only if it did not raise and its normalised output equals the reference. Normalisation collapses line endings, trailing whitespace and repeated blank lines, so cosmetic variation does not fail a correct answer. Because the comparison is on runtime output rather than source text, a student who writes a comprehension where the instructor wrote a loop is accepted. This satisfies G2 and it is what makes generated exercises safe: the model proposes a task, the kernel decides the outcome.

5.3 The help ladder and adaptive difficulty

When a run fails, the student is told whether the cell raised an error or produced the wrong result, and the raw output is shown. Hints are revealed one at a time and only on request. A Reveal the answer control is hidden initially and appears once the learner has either taken a hint or had a run fail, so it is never the first thing offered. Taking a hint or failing a run clears the first try flag, which is recorded but does not reduce the experience points awarded for eventually solving the cell; only revealing the answer forfeits them. This is the graduated feedback of G3: the student is never stuck, never handed the answer without asking, and never punished twice for asking. A difficulty bias of easier, normal or harder is injected into the generation prompt, and on the harder setting a failed run stops offering the reveal at all, so a student who asked for a harder session does not get an early escape hatch. It is the one place where the learner regulates their own challenge level, the autonomy component of motivation in self determination theory [14].

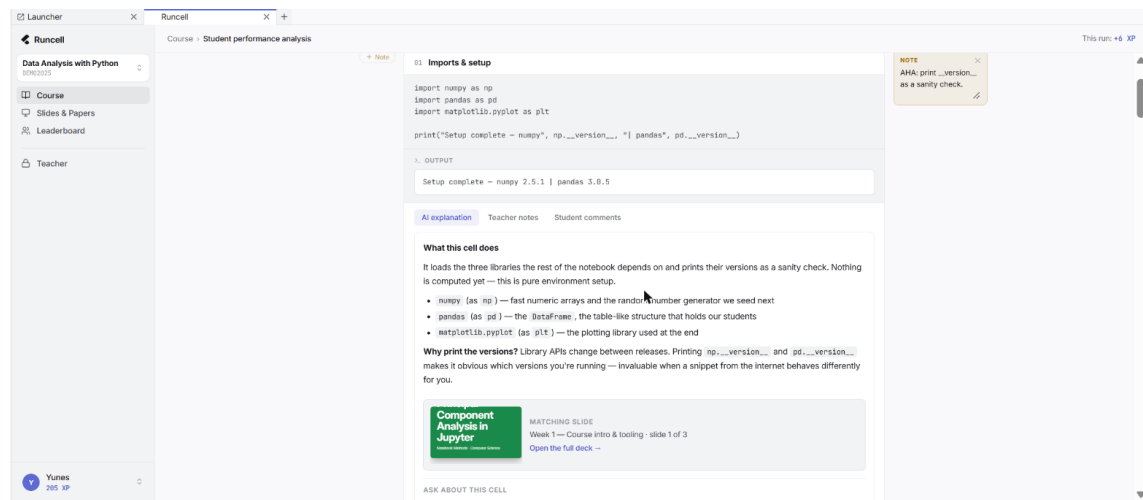


Fig. 4. Explain mode. Every cell carries an AI explanation and tutor chat, the instructor's note with the week's slide inline, the peer comment thread, and private margin notes.

5.4 Explain mode and supporting surfaces

Learn mode is demanding, and it is the wrong mode for a first encounter with unfamiliar material. Explain mode is the reading counterpart: the notebook becomes a sequence of cards, one per cell, each showing the code, its output, and a tabbed context panel with three sources of knowledge. The first is an AI explanation of what the cell does, plus a tutor chat scoped to that one cell; the second is the instructor's note with the week's slide embedded inline; the third is the peer comment thread. Embedding the slide beside the cell rather than behind a separate materials button came from formative feedback, since students had been reading code in one place and slides in another.

On wide screens each card is flanked by margin columns for private sticky notes. These are the one piece of content in the system that is strictly private: they persist to the student's own account, survive reload, and are readable by nobody else, not classmates and not the instructor. The intent is to give the student a place to record the moment they understood something, the self explanation behaviour [4] shows to be beneficial, and to make that place safe enough that they use it.

Around the two modes sit a course home with a week timeline, a slide and paper reader with quizzes and flashcards whose review date is pushed out by how the learner rated the card, a course map of the dependency structure between topics, and a leaderboard with a friends screen. One surface defines the system's floor: Bring your own notebook lets a user upload an arbitrary .ipynb file and learn it with no course and no instructor, keeping it on the device in IndexedDB rather than in their account, so the file itself is never transmitted.

6 THE TEACHING SIDE

An instructor creates a course, receives an invite code, and publishes weeks of notebooks and slides that can be locked or unlocked to control pacing. Per cell notes are authored in Markdown, and generated exercises can be overridden with hand written tasks, which is how G1 preserves instructor control: generation is the default, not the only option.

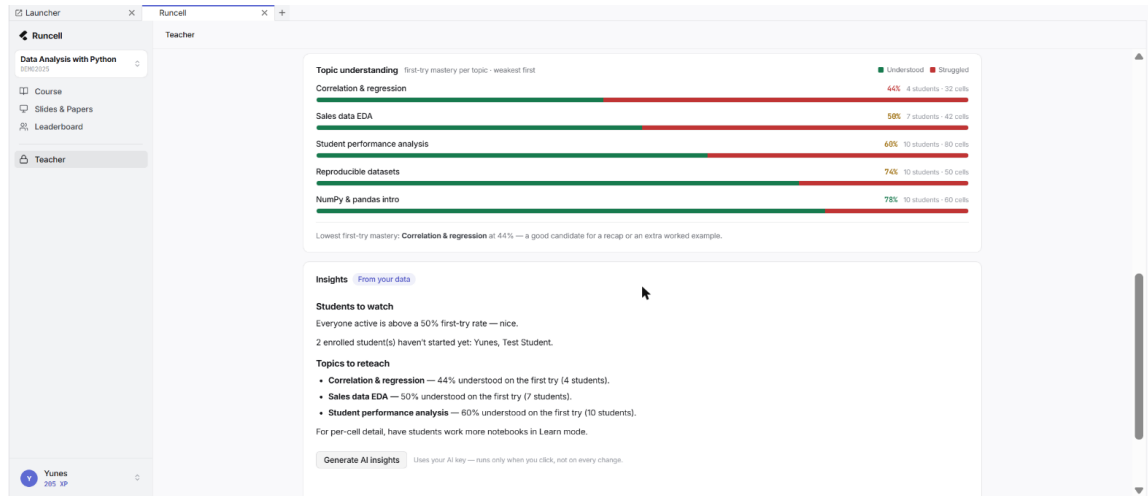


Fig. 5. Instructor dashboard. Per topic first try mastery, weakest first, with counts of students and cells but no names attached to any failure. Below it the algorithmic summary updates continuously, while the AI report stays behind an explicit button.

6.1 Analytics without attribution

The dashboard answers three questions. Which cells does the class fail? Which topics are weak? Is the class progressing? It answers them from two aggregate stored procedures and one anonymous table.

Per cell difficulty comes from the `cell_attempts` table, which has no user column at all: a row records the notebook, the cell index, the course, whether the attempt succeeded and which attempt number it was. The instructor sees, per cell, a struggle score computed as one minus the first try rate, ordered worst first. This is the mechanism behind G5, and the important property is that it is enforced by the shape of the data rather than by the interface: even a database administrator running arbitrary SQL cannot recover who failed a given cell.

Per topic understanding comes from `notebook_submissions`, which does carry a user reference, aggregated by notebook title into a stacked bar of understood versus struggled proportions; the roster shows enrolled students ranked by course experience points, with their notebooks completed and their overall first try rate. The boundary is therefore drawn at the cell, not at the student: an instructor can see that someone is finding the course hard, which is what lets them offer help, but cannot see *which cell* any named student failed, because that is the record carrying no identity. The dashboard labels its own header *Aggregates only*.

The dashboard also writes an algorithmic summary of the aggregates continuously, without any model call; a separate button sends the same aggregates to the LLM for a prose report. The AI report never runs automatically, because live data changes would otherwise trigger a paid model call on every submission, and reports are cached with a signature of the data they came from, so when the numbers move the report is marked stale rather than silently misleading the instructor. Six tables are published to the realtime feed, so a student joining, submitting or commenting repaints the dashboard and the affected Explain mode threads without a reload.

Table 2. Visibility matrix. Failure data is anonymous at the storage layer; social data requires explicit and revocable consent.

Data	Owner	Peers	Instructor
Private cell notes	yes	no	no
Per cell attempts	n/a	no	aggregate only
Notebook submissions	yes	no	yes
Experience points	yes	if opted in	yes
Leaderboard rank	yes	if opted in	yes
Friend statistics	yes	if mutual	no
Peer comments	yes	yes	yes

7 PRIVACY AND SECURITY MODEL

Table 2 states who can see what. The pattern is an asymmetry: everything that motivates is identified and consensual, everything that diagnoses is anonymous.

Row level security everywhere. Every table has policies enabled and explicit grants. The client holds only an anonymous key plus the user’s own token, so the key can be published safely and authorization is decided by PostgreSQL for every query. Where a query legitimately needs to read across users, such as ranking a leaderboard, it is a stored procedure that runs with elevated rights but first checks that the caller is enrolled. That pattern also resolved a policy recursion cycle in which the courses policy consulted enrolments and the enrolments policy consulted courses.

Opt in leaderboards, scoped per course. A student appears only if they set the opt in flag, and rankings use points earned in that course rather than a lifetime total, so work in one course does not inflate standing in another. This required tagging every point event with its course, a schema change made specifically to fix the leak.

Mutual consent for peer comparison. Friend links are directional. One direction means “I have chosen to share my statistics with you” and reveals nothing in return; statistics become visible only when both directions exist, and either side can withdraw at any moment. A stored procedure also lets a user delete their own account, which cascades every row they own.

8 IMPLEMENTATION AND VALIDATION

8.1 Degradation ladder

G6 is an explicit ladder, each rung a supported state rather than an error. With *no AI key*, every exercise becomes the deterministic write the cell type and explanations fall back to written defaults; courses, verification, points, leaderboards and analytics are unaffected, because none of them depend on the model. With *no backend*, the extension runs locally: uploads go to IndexedDB and progress is session scoped, but the learning loop is intact because verification needs only the kernel. With *no kernel*, verification degrades to comparing the student’s source against the reference source, the only rung where G2 is sacrificed.

A verification loop built on kernel execution also inherits the fragility of environment management. For a period during development no cell would execute, because the kernel specification named an interpreter resolved from the system path rather than the project’s virtual environment. The setup scripts now bind the kernel to that environment by absolute path.

8.2 Validation

Backend correctness is checked by an automated suite that runs against the live database using the same client library as the application, covering 23 sections and 76 assertions and exiting non zero on failure. It spans authentication, profiles, course creation and invite enrolment, the instructor dashboard aggregates, points and the opt in leaderboard, per cell attempts, notebook and comment synchronisation, the generated exercise cache, the friend life cycle from request through mutual sharing to withdrawal, account deletion, a live realtime event and a live model completion. On the run taken for this report 72 assertions passed, none failed and four skipped: the on device upload, which is IndexedDB and therefore verifiable only in a browser; the destructive delete round trip and the realtime publication check, both of which need a direct Postgres connection string rather than the public key; and sign up, which the shared project throttles by an email rate limit. The realtime delivery check is timing sensitive and times out against a cold socket on the free tier, so it needs a warm connection to pass.

Two properties matter. The suite exercises the real deployment rather than mocks, so a row level security policy that is wrong in production fails the test; and it asserts negative cases, such as a non instructor being refused the dashboard call and a second user being unable to read a student's private notes, which is the only way to test a privacy claim.

9 DISCUSSION

9.1 What is new

First, *output equivalence verification for generated exercises*. Prior work showed that models can generate programming exercises but need review before use [16], and that automated grading works when an instructor writes the tests [1]. Runcell combines them by taking the answer key from the instructor's own already executed cell, so authoring cost drops to zero while the acceptance criterion becomes more permissive, not less.

Second, the *privacy asymmetry* of Section 7. Learning analytics dashboards conventionally present per student detail, and the ethical literature has argued at length against exactly that [17]. Runcell splits the telemetry: data that motivates is identified and opt in, data that diagnoses is anonymous by construction. Because the anonymity lives in the schema rather than in an access control rule, it survives changes to the interface and to policy.

Third, the *degradation ladder*: a system whose learning loop stops when a paid API key expires cannot be given to a class. Each of these carries an accepted cost. Being an extension buys a kernel with the course libraries but requires an installation step; having no application server keeps authorization in one place but ties the system to one platform; and anonymous attempt records buy diagnostic honesty at the price of individual follow up.

9.2 Limitations and future work

The most significant limitation is that we have no controlled evaluation of learning outcomes. The system was validated technically and shaped by formative feedback, but no study compares students using Runcell against students reading the same notebooks conventionally, so every pedagogical claim here rests on the cited literature and on the design following from it rather than on measured effect.

Output equivalence has a real failure mode: cells whose result is a plot, a random sample or a wall clock timing do not produce stable comparable text, and although the system captures images separately it does not compare them. Generation quality varies with the model, and the validation layer turns poor generations into safe fallbacks rather than good exercises, so a weak model produces a monotonous session of write the cell tasks. Finally, the deployment

depends on one hosted backend platform and has been exercised at the scale of a seeded course rather than a live cohort.

The clearest next step is a controlled study measuring retention and transfer against conventional notebook reading, with the anonymous attempt data as the natural instrument since it is already aggregate and needs no further consent. Structural comparison of plot outputs would extend verification to visualisation heavy material, and an instructor approval queue for generated exercises would let a course lift quality where it matters.

10 CONCLUSION

Runcell addresses a widespread failure of computational notebooks as a teaching medium: the finished notebook invites recognition where learning requires retrieval. Rather than asking instructors to rewrite their material as exercises, it derives exercises from what they already have, and rather than trusting a language model to grade, it lets the Python kernel decide by comparing what the student’s code does against what the instructor’s cell did. Around that loop sit a course model, a progression system whose social features are opt in and revocable, and an instructor dashboard whose per cell failure signal is anonymous in the schema rather than in the interface. Its pedagogical effect remains to be measured.

CONTRIBUTIONS

Runcell was built by Yunes Mahan and Johannes Specht as a two person project, with the work split evenly. We met weekly to plan the next increment, settle the design decisions justified in this report, and divide that week’s tasks into two comparable halves. Neither of us owned a subsystem outright: both of us worked across the TypeScript frontend, the Postgres schema and its row level security policies, and the test suite, and each week’s output was reviewed by the other before it was merged. This report is written and submitted individually, as the course requires.

AVAILABILITY

Source code, database migrations, setup scripts and the automated test suite are available at

<https://github.com/yunes-mahan/NotebookMind>

The repository README documents system requirements, a one command setup for Windows, macOS and Linux, and disposable accounts for a seeded demonstration course.

REFERENCES

- [1] Douglas Blank, David Bourgin, Alexander Brown, Matthias Bussonnier, Jonathan Frederic, Brian Granger, Thomas L. Griffiths, Jessica B. Hamrick, Kyle Kelley, M. Pacer, Logan Page, Fernando Pérez, Benjamin Ragan-Kelley, Jordan W. Suchow, and Carol Willing. 2019. nbgrader: A Tool for Creating and Grading Assignments in the Jupyter Notebook. *The Journal of Open Source Education* 2, 16 (2019), 32. <https://doi.org/10.21105/jose.00032>
- [2] Annabell Broucker, Sven Judel, René Roepke, Nikolina Mihailovska, and Ulrik Schroeder. 2022. Gamifying JupyterLab to Encourage Continuous Interaction in Programming Education. In *Games and Learning Alliance (GALA 2022). Lecture Notes in Computer Science*, vol. 13647. Springer, 321–326. https://doi.org/10.1007/978-3-031-22124-8_32
- [3] Annabell Broucker, Sven Judel, and Ulrik Schroeder. 2022. Integration von Gamification und Learning Analytics in Jupyter. *eleed, Issue 14* (2022). <https://www.eleed.de/archive/14/5533>
- [4] Michelene T. H. Chi, Nicholas de Leeuw, Mei-Hung Chiu, and Christian LaVancher. 1994. Eliciting Self-Explanations Improves Understanding. *Cognitive Science* 18, 3 (1994), 439–477. https://doi.org/10.1207/s15516709cog1803_3
- [5] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (2024), 56–67. <https://doi.org/10.1145/3624720>

- [6] Yuemeng Du, Andrew Luxton-Reilly, and Paul Denny. 2020. A Review of Research on Parsons Problems. In *Proceedings of the Twenty-Second Australasian Computing Education Conference (ACE '20)*. ACM, 195–202. <https://doi.org/10.1145/3373165.3373187>
- [7] Barbara J. Ericson, Paul Denny, James Prather, Rodrigo Duran, Arto Hellas, Juho Leinonen, Craig S. Miller, Briana B. Morrison, Janice L. Pearce, and Susan H. Rodger. 2022. Parsons Problems and Beyond: Systematic Literature Review and Empirical Study Designs. In *Proceedings of the 2022 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '22)*. ACM, 191–234. <https://doi.org/10.1145/3571785.3574127>
- [8] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. <https://doi.org/10.3102/003465430298487>
- [9] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. 2016. Jupyter Notebooks: A Publishing Format for Reproducible Computational Workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas. Proceedings of the 20th International Conference on Electronic Publishing*. IOS Press, 87–90. <https://doi.org/10.3233/978-1-61499-649-1-87>
- [10] Mark Liffiton, Brad E. Sheese, Jaromir Savelka, and Paul Denny. 2023. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. ACM, 1–11. <https://doi.org/10.1145/3631802.3631830>
- [11] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2019. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. In *Proceedings of the 16th International Conference on Mining Software Repositories (MSR '19)*. IEEE, 507–517. <https://doi.org/10.1109/MSR.2019.00077>
- [12] Henry L. Roediger and Jeffrey D. Karpicke. 2006. The Power of Testing Memory: Basic Research and Implications for Educational Practice. *Perspectives on Psychological Science* 1, 3 (2006), 181–210. <https://doi.org/10.1111/j.1745-6916.2006.00012.x>
- [13] Adam Rule, Aurélien Tabard, and James D. Hollan. 2018. Exploration and Explanation in Computational Notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. ACM, 1–12. <https://doi.org/10.1145/3173574.3173606>
- [14] Richard M. Ryan and Edward L. Deci. 2000. Self-Determination Theory and the Facilitation of Intrinsic Motivation, Social Development, and Well-Being. *American Psychologist* 55, 1 (2000), 68–78. <https://doi.org/10.1037/0003-066X.55.1.68>
- [15] Michael Sailer and Lisa Homner. 2020. The Gamification of Learning: A Meta-analysis. *Educational Psychology Review* 32, 1 (2020), 77–112. <https://doi.org/10.1007/s10648-019-09498-w>
- [16] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research (ICER '22)*. ACM, 27–43. <https://doi.org/10.1145/3501385.3543957>
- [17] Sharon Slade and Paul Prinsloo. 2013. Learning Analytics: Ethical Issues and Dilemmas. *American Behavioral Scientist* 57, 10 (2013), 1510–1529. <https://doi.org/10.1177/0002764213479366>
- [18] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285. https://doi.org/10.1207/s15516709cog1202_4